

0.82.1. Un mur qu'on ne traverse pas : le M — un labyrinthe

Des M tout autour et un mur au milieu : un labyrinthe. La pièce P est derrière le mur ; il faut faire le tour pour l'attraper.

1. Ce qu'il faut comprendre

C'est le 0.82, avec plus de murs : un cadre de M tout autour, et le mur du milieu, ouvert en bas. La pièce P (le 0.81) attend derrière.

Ce qui est nouveau ici : dessiner des murs avec `texte`. Une ligne de vingt M, c'est `texte(0, 2, "MMMMMMMMMMMMMMMMMMMM")` ; : un seul appel. Les côtés, eux, se posent un par un dans une boucle.

Le déplacement ne change pas : c'est `lire()` qui fait tout le travail. Il ne sait pas où sont les murs à l'avance : il **regarde l'écran**. Ajoute des M où tu veux, le A ne les traversera pas.

Le chemin : descendre sous le mur (lignes 11 à 13), passer à droite, remonter jusqu'au P.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction `chaque\(\)` — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction `bouton\(\)` — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction `lire\(\)` — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction `effacer\(\)` — effacer des cases](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1131 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	66 o	← la console, à l'allumage — la mise en route de la console
chaque_minuteur()	124 o	← <code>main()</code> — une fonction fournie par le langage
<code>main()</code>	343 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> , <code>EcrireTexte</code> , <code>EcrireNombre</code> — attend de pouvoir écrire une case
AdresseCase	10 o	← <code>poser()</code> , <code>lire()</code> , <code>effacer()</code> — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← <code>effacer()</code> — efface des cases
EcrireTexte	10 o	← <code>texte()</code> — écrit un texte, lettre par lettre
EcrireNombre	33 o	← <code>nombre()</code> — écrit un nombre en chiffres

```

CopierTuiles      85 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            288 o ← CopierTuiles – 18 lettres sur 43
"MMMMMMMMMMMMMM..." 20 o ← texte() – un texte gravé
"MMMMMMMMMMMMMM..." 20 o ← texte() – un texte gravé
"SCORE"           5 o ← texte() – un texte gravé
Lettres gravées : ACEMOPRS0123456789
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
poser()          x 6    68 o sur place, dans main() + AdresseCase (10 o)
texte()          x 3    33 o sur place, dans main() + EcrireTexte (10 o)
bouton()         x 4    20 o sur place, dans main() + LireManette (37 o)
effacer()        x 1    15 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
lire()           x 1    11 o sur place, dans main() + AdresseCase (10 o)
nombre()         x 1    10 o sur place, dans main() + EcrireNombre (33 o)
image()          x 1    3 o sur place, dans main() + AttendreImage (28 o)
images()         x 1    2 o sur place, dans chaque_minuteur()
Tes fonctions appelées :
chaque_minuteur() x 1    10 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- LES VARIABLES ----

#include <texte>      // écrit un texte à l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
#include <effacer>    // efface des cases, ou tout le fond
#include <nombre>     // écrit un nombre en chiffres

```

2. UNE VARIABLE — X

```
uint8_t x = 3;      // le A : il part de (3, 5), à gauche du mur du milieu

```

3. UNE VARIABLE — Y

```
uint8_t y = 5;

```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;     // la case d'arrivée, calculée avant chaque pas (le 0.82)

```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;

```

6. UNE VARIABLE — SCORE

```
uint8_t score = 0;

```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- LE LABYRINTHE : tout en lettres M ----
    //
    // ligne 2 : M M M M M M M M M M M M M M M M M le haut
    // ligne 3 : M M M M M M M M M M M M M M M M M
    // ligne 5 : M A M P M A : départ, P : la pièce
    // ... M M M M M M M M M M M M M M M M M
    // ligne 10 : M M M M M M M M M M M M M M M M M le mur du milieu s'arrête ici...
    // ligne 11 : M M M M M M M M M M M M M M M M M ...les lignes 11 à 13
    // ligne 13 : M M M M M M M M M M M M M M M M M sont libres : on passe
    // ligne 14 : M M M M M M M M M M M M M M M M M le bas

    // Le haut et le bas : une ligne de 20 M, en UN seul texte.
    texte(0, 2, "MMMMMMMMMMMMMMMMMMMMMMMM");
    texte(0, 14, "MMMMMMMMMMMMMMMMMMMMMMMM");

    // Les deux côtés : un M par ligne, à gauche (colonne 0) et à droite (colonne 19).
    for (uint8_t l = 3; l < 14; l++) {
        poser(0, l, ALPHABET[12]);
        poser(19, l, ALPHABET[12]);
    }

    // Le mur du milieu, en colonne 10, des lignes 3 à 10 seulement : ouvert en bas.
    for (uint8_t l = 3; l <= 10; l++) {
        poser(10, l, ALPHABET[12]);
    }

    poser(16, 5, ALPHABET[15]); // la pièce P, de l'autre côté du mur
    poser(x, y, ALPHABET[0]); // le A, à son départ
    texte(0, 17, "SCORE");

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();
        if (chaque(250)) { // au plus un pas tous les 250 ms (le 0.77)

            // ÉTAPE 1 : calculer la case d'ARRIVÉE, sans encore bouger.
            // On part de la case actuelle : si aucune flèche n'est tenue,
            // l'arrivée reste la case où l'on est.
            nx = x;
            ny = y;
            if (bouton(DROITE) && x < 19) { nx = x + 1; } // une colonne à droite
            if (bouton(GAUCHE) && x > 0) { nx = x - 1; } // une colonne à gauche
            if (bouton(BAS) && y < 16) { ny = y + 1; } // une ligne plus bas
            if (bouton(HAUT) && y > 0) { ny = y - 1; } // une ligne plus haut

            // ÉTAPE 2 : REGARDER ce qu'il y a sur la case d'arrivée, avec lire() :
            //
            // lire(nx, ny) != ALPHABET[12]
            // | |
            // | | +-- le M, la lettre des murs
            // | +----- != : « n'est pas »
            // +----- ce que montre l'écran à la case d'arrivée
            //
            // Exemple : le A est en (9, 8), la flèche droite est tenue.
            // L'arrivée est (10, 8). lire(10, 8) rend le M du mur :
            // « n'est pas un M » est FAUX → le A ne bouge pas.
            if (nx != x || ny != y) { // une flèche est tenue (|| : OU)
                if (lire(nx, ny) != ALPHABET[12]) {

                    // ÉTAPE 3 : ce n'est pas un mur, on avance.
                    effacer(x, y, 1); // efface le A à l'ancienne case
                    x = nx; // la case d'arrivée devient
                    y = ny; // la case du A
                }
            }
        }
    }
}
```

```

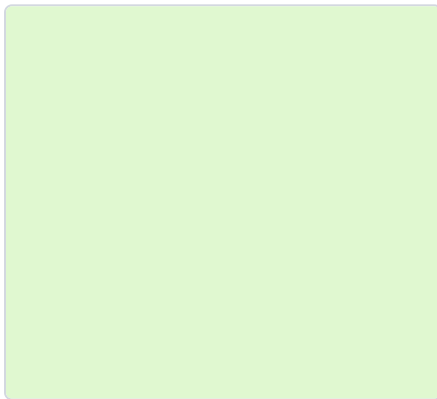
        poser(x, y, ALPHABET[0]);    // pose le A à la nouvelle case
    }
}
}

// La pièce, en (16, 5) : le A est dessus ?
if (x == 16 && y == 5) {
    if (score == 0) {                // seulement la première fois :
        score = 1;                  // le score passe à 1, une fois pour toutes
    }
}
nombre(6, 17, score);
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



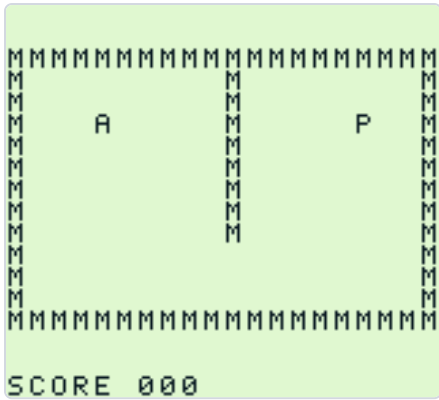
la toute première image — la console vient de démarrer — image 1



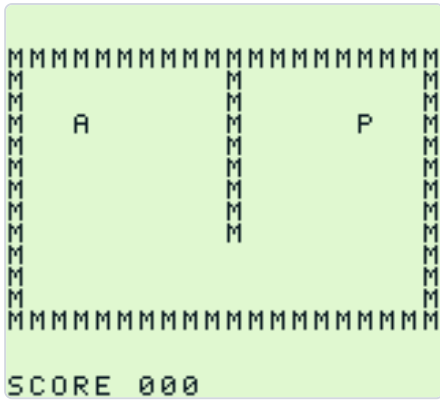
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



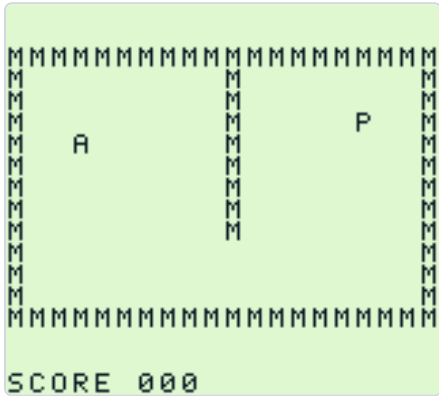
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran n'a pas bougé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un labyrinthe de M ; le A doit descendre, passer sous le mur, et remonter jusqu'au P.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le mur du milieu l'arrête
- ✓ les murs du cadre aussi : jamais dans un M
- ✓ il est passé de l'autre côté

Ensuite : **0.82.2. La fonction `note()` — jouer une note** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++